# Amazon EventBridge Overview

Explore Amazon EventBridge as a core AWS serverless event bus that facilitates event-driven architectures. Learn how to route events from AWS services and custom sources, define rules and filters, use scheduled events, and implement best practices for security and monitoring. Understand event schema management, fanout and choreography patterns, idempotency, and cross-account delivery to build scalable, decoupled applications.

In this lesson, we explore Amazon EventBridge as a foundational service for building event-driven applications on AWS. EventBridge enables developers to create scalable, loosely coupled systems by routing events from AWS services, SaaS applications, and custom sources. We focus on its architecture, event routing logic, developer integrations, security best practices, and monitoring capabilities.

## Introduction to Amazon EventBridge

Amazon EventBridge is a serverless event bus service designed to simplify communication between application components using events. It ingests events from AWS services, custom applications, and integrated SaaS partners, and routes them to targets like AWS Lambda, Step Functions, and SNS.

Ask

Some documentation or exam questions may refer to "Amazon CloudWatch Events". This is simply the earlier name for what is now known as EventBridge. The service has since evolved with added capabilities like schema discovery, partner event sources, and advanced filtering.

## Core concepts and components

An EventBridge architecture consists of several key elements:

- **Event buses:** Channels for routing events. There are three types: the default event bus, custom event buses, and partner event buses.
- **Source:** The origin of the event, such as a custom application, an integrated SaaS platform, or an AWS service that emits events.
- **Targets:** AWS services that respond to events, such as Lambda functions, Step Functions, SQS queues, or SNS topics.
- **Events:** JSON-formatted messages describing a state change or action. Events typically include a `source`, `detail-type`, `detail`, and `time`.
- **Rules:** Define matching patterns and specify one or more targets to route matching events to. To programmatically configure routing, we can define a rule and specify one or more targets. For example, here's how we can create a rule that listens for high-value orders and routes them to a Lambda function:

main.py Python 3.10.4

```
import boto3

events = boto3.client('events')
lambda_arn = "arn:aws:lambda:<Region>:<Account-ID>:function:ProcessOr

# Create the rule
events.put_rule(
    Name="HighValueOrderRule",
    EventPattern='''
    {
```

Ask

```
        "detail-type": ["order_placed"],
        "detail": {
```

Create an EventBridge rule using Boto3

- **EventBridge pipes:** EventBridge pipes connect a source and a target and support point-to-point integration. They allow us to filter and enrich the events sent to a destination. The following illustration gives an overview of how a working EventBridge pipe looks:

EventBridge pipes

These components together enable event producers and consumers to remain loosely coupled, increasing system flexibility and resilience.

## How event routing works

When a service specified as the source in an event pipe emits an event, it's sent to an event bus. Rules apply pattern matching and forward the event to one or

Ask

more targets. Rule supports both simple matching ( like filtering by `detail-type`) and complex conditions using JSON path syntax and operators like `prefix`, `numeric`, or `anything-but`.

Here's an example filter that routes only high-value orders:

```
{
  "detail-type": ["order_placed"],
  "detail": {
    "amount": [{"numeric": [">=", 100]}]
  }
}
```

Route only high-value orders

These filters help avoid unnecessary invocations of downstream services like Lambda, thereby reducing cost and improving efficiency.

## Automating events without a source

Imagine we need to run a cleanup job every day at midnight to archive stale records or remove temporary files from an S3 bucket. We don't want to rely on external schedulers or manage cron jobs on EC2. Instead, we can configure a scheduled rule in Amazon EventBridge that automatically invokes a Lambda function at the desired time.

To define when the rule should run, EventBridge allows us to use two types time-based expressions.

Ask

- `rate()`: We can use this type of expression for fixed intervals, such as every five minutes or every six hours.
- `cron()`: We can use this type of expression for more specific calendar-based schedules, such as triggering the job at midnight every day,

In this case, the we can use `cron(0 0 * * ? *)` expression to tell EventBridge to invoke the target exactly at **00:00 UTC** each day. With a scheduled rule configured this way, the cleanup task runs automatically without any external event triggering it. The rule acts as the event source, simplifying automation and keeping our architecture fully serverless.

## Emitting custom events

EventBridge isn't just for AWS services or SaaS providers, our own applications can also act as event source. This is especially powerful when we want to decouple components of our system. For example, instead of calling downstream services directly, our application can emit an event like `"order_placed"` or `"user_signed_up"` and let EventBridge handle routing and fanout.

To emit a custom event, we can use the AWS SDK. Here's how we can do it:

main.py Python 3.10.4

```
import boto3
client = boto3.client('events')

client.put_events(
    Entries=[{
        'Source': 'my.app',
        'DetailType': 'orderPlaced',
        'Detail': '{"orderId": "12345"}',
        'EventBusName': 'default'
    }]
)
```

Ask

EventBridge integration using Boto3

We create a client, define the event details like the source and payload, and then call the API to send the event to the specified event bus. Once emitted, these events can trigger any service configured as a target based on matching rules.

## EventBridge Schema Registry

In event-driven systems, producers and consumers often evolve independently. This flexibility is powerful, but it can lead to problems when services misunderstand the structure of events, especially as systems grow. A missing field, a wrong data type, or a misnamed attribute can cause downstream failures that are hard to debug.

To solve this, Amazon EventBridge provides a *Schema Registry*, a feature that helps teams define, share, and validate the structure of events.

EventBridge can automatically generate these definitions using a feature called schema discovery. When enabled, EventBridge inspects the events sent to a custom event bus and automatically creates a JSON schema based on the event's structure.

Here's a quick example of enabling discovery using Boto3:

main.py Python 3.10.4

```
import boto3

client = boto3.client('schemas')

client.put_discoverer(
    SourceArn='arn:aws:events:<Region>:<Account-ID>:event-bus/my-
)
```

Ask

Enabling schema discovery

These schemas are saved in a special registry so other teams or services can rely on a consistent event format. We can then browse these schemas in the AWS Console or download them using the CLI. For example, with generated bindings, our Lambda function can safely access event fields like this:

main.py Python 3.10.4

```
from my_app_schemas import OrderPlaced

def lambda_handler(event, context):
    order = OrderPlaced(event)
    print(order.orderId)  # autocomplete and type safety
```

Using Schema Registry in Lambda function

In the example, the Lambda function receives a raw event and wraps it using `OrderPlaced(event)`, turning it into a strongly-typed object. This allows us to access fields such as `order.orderId` with autocompletion and type safety, making our code more robust and easier to maintain.

Ask

## Fanout pattern and choreography

EventBridge supports architectural patterns like fanout and choreography, which are key to building scalable, event-driven systems that remain loosely coupled.

Fanout occurs when a single event triggers multiple downstream processes. For example, when an `order_placed` event is emitted, we might want to notify the customer via email, update inventory, and trigger a billing process, all in parallel. With EventBridge, this is done by defining multiple rules that match the same event and route it to different targets like Lambda, SNS, or Step Functions. Each target processes the event independently, allowing the system to scale horizontally and respond quickly. This approach avoids tight coupling so no component needs to know who else is involved.

Choreography takes this further. Rather than using a central orchestrator to control the flow (like Step Functions), each service acts independently and listens for events it cares about. For instance, the `order_placed` event is emitted once, and the billing service, inventory service, and notification service each subscribe to it through their own rules.

Both of these patterns can coexist. We can use fanout when one event should trigger multiple actions, and use choreography when independent services need to respond to events in a decoupled, asynchronous manner.

## Asynchronous and stateless integration

Amazon EventBridge uses an asynchronous communication model. When an event matches a rule and is sent to a target like a Lambda function or Step Function, EventBridge does not wait for the target's execution to complete. It simply delivers the event and moves on. This means EventBridge does not track whether the target succeeded or failed in processing the event.

This design choice helps decouple services and improve scalability, but it a implications for observability and error handling:

Ask

- EventBridge cannot detect or report the result of the target's processing. If a Lambda function fails, EventBridge won't know — unless additional configuration is in place.
- To handle undelivered events, we can attach a Dead Letter Queue (DLQ) to an EventBridge rule. If the target is unreachable after all retries, the event is sent to the DLQ.
- For deeper visibility into what happens after delivery, use CloudWatch Logs on the target service, or configure Lambda Destinations to capture success or failure outcomes explicitly.

This behavior makes EventBridge ideal for loosely coupled, event-driven systems where each component processes messages independently and failure handling is managed through external monitoring or retry strategies.

## Delivery guarantees and latency considerations

Amazon EventBridge offers at-least-once delivery of events to targets like Lambda, Step Functions, and SNS, ensuring that each event is delivered at least once, but possibly more in rare retry scenarios.

However, it's important to understand where EventBridge fits in the broader AWS messaging ecosystem. EventBridge is not optimized for high-throughput or low-latency event delivery. It's suitable for typical application-level events, such as "order placed" or "user signed up", but not ideal for scenarios requiring millisecond-level processing or handling thousands of messages per second.

The table below illustrates the differences between EventBridge and other AWS messaging and streaming services which will help us decide when to use EventBridge and when to opt for other AWS services:

Ask

| Feature | EventBridge | Amazon SNS | Amazon SQS | Amazon Kinesis/MSK |
|---|---|---|---|---|
| **Delivery guarantee** | At-least-once | At-least-once / Best-effort | At-least-once | At-least-once |
| **Latency** | High (seconds) | Low (milliseconds) | Moderate (milliseconds) | Very low (milliseconds) |
| **Throughput** | Moderate | High | Very high | Very high |
| **Replay Capability** | No | No | No | Yes |
| **Ordering Support** | No | No | FIFO queues available | Yes |
| **Use Case Fit** | Event routing and orchestration | Fanout notifications | Workload buffering | Real-time analytics, streaming |

## Idempotency in event processing

As discussed earlier, EventBridge guarantees at-least-once delivery. Therefore, consumers must implement idempotent processing to handle duplicate events safely. This is especially important for Lambda targets.

Imagine a payment processing Lambda that charges a user's credit card based on an incoming event. If this Lambda is triggered twice for the same event due to a retry, the customer could be charged twice unless the function checks whether the payment was already processed.

To prevent this, we can implement an idempotency check using a DynamoDB table to track processed event IDs. The code below demonstrates this:

main.py

Python 3.10

Ask

```
import boto3
import json
```

```
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('ProcessedEvents')

def lambda_handler(event, context):
    event_id = event['id']

    # Check if this event ID already exists
    response = table.get_item(Key={'event_id': event_id})
    if 'Item' in response:
```

Using idempotency check

By storing processed event IDs in DynamoDB, the function ensures that only the first invocation has an effect. This pattern is crucial for protecting against accidental duplication in distributed systems.

## IAM permissions and cross-account event delivery

To successfully publish events to Amazon EventBridge, our application must have the appropriate IAM permissions. At a minimum, we need to grant the `events:PutEvents` action for the target event bus. This is common when a Lambda function or EC2 instance emits custom events.

Here's a basic IAM policy that grants an application permission to put events on the default event bus:

index.yaml YAML

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "events:PutEvents",
      "Resource": "arn:aws:events:<Region>:<Account-ID>:event-bus/defa
    }
```

Ask

```
    ]
  }
```

Policy to publish events

This policy restricts access to a specific event bus, which follows the principle of least privilege. Replace `default` with the name of a custom bus if needed.

EventBridge also supports secure cross-account event delivery. For instance, in a multi-account setup, Account B may need to send events to Account A. To enable this, Account A (the receiver) must explicitly allow the sender using a resource-based policy on the event bus.

Below is a sample policy applied to Account A's custom bus (`my-shared-bus`), allowing Account B (ID `123456789012`) to put events:

**index.yaml** YAML

```
{
  "Statement": [
    {
      "Sid": "AllowAccountB",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<AccountB-ID>:root"
      },
      "Action": "events:PutEvents",
      "Resource": "arn:aws:events:<Region>:<AccountA-ID>:event-bus/my-
    }
  ]
}
```

Ask

Resource-based policy for event bus

We can apply this policy using the following CLI command:

main.sh Shell

```shell
aws events put-permission \
  --event-bus-name my-shared-bus \
  --action events:PutEvents \
  --principal <Requesting-entity-ARN> \
  --statement-id AllowAccountB
```

Add resource-based policy to an event bus

This configuration is essential for scenarios like centralized audit pipelines or SaaS multi-tenant architectures

## Monitoring and error handling

Amazon EventBridge integrates with CloudWatch for monitoring rule matches, failed deliveries, and invocation metrics. We can configure dead-letter queues (DLQs) for undelivered events.

When EventBridge targets a Lambda function, we may also configure Lambda Destinations to route successful or failed executions to EventBridge, SNS, or SQS. This is distinct from EventBridge DLQs and gives developers more control over post-processing workflows.

Ask

The table below summarizes how an EventBridge rule interacts with a Lambda function and what configurations and outcomes apply at each step of the event

delivery process:

| Applies to | EventBridge Rule | Lambda Function |
|---|---|---|
| **Rule-to-target Delivery** | Delivers matching events to the target (Lambda) | Invoked when an event is received |
| **Lambda Execution Outcome** | Not aware of function's result | Returns success or error, logged in CloudWatch |
| **Set On** | EventBridge Rule | Lambda Function |
| **Use Case** | Filtering and routing specific events | Processing event logic or triggering actions |
| **Delivery Failure** | Tracked in EventBridge Dead Letter Queue (optional) | Tracked in CloudWatch Logs and via retries |
| **Post-success/Failure logic** | Not applicable | Handled via code or Step Functions chaining |

# Conclusion

Amazon EventBridge provides a powerful foundation for building scalable, event-driven applications on AWS. By decoupling producers and consumers through

Ask